Constraint-Level Evaluation and Repair for Text-to-Image Prompt Following

A grammar-based system for measuring and repairing partial prompt-following failures in T2I generation.

Yiling Huang

CS348K Final Project

Stanford University

Problem: T2I failures are partial

A generated image can look plausible while missing specific prompt requirements.

Prompt example

A realistic image of exactly 4 dice inside a cup on a shelf full of books and decorations.

Why image-level quality is not enough

- Cardinality: wrong count
- Spatial relation: correct

Main idea: evaluate and repair at the constraint level, not only at the whole-image level.

Project question and success definition

Core question:

How can we make partial T2I prompt-following failures measurable and repairable?

Practical extension:

Can a VLM checker scale this process by identifying repair targets automatically?

Success definition:

1. **Measurability**: expose meaningful partial failures instead of only image-level pass/fail.
2. **Repair usefulness**: turn failed or ambiguous requirements into actionable repair signals.
3. **Practicality/stability**: Remain stable across prompt families, T2I models, and human/VLM checking.

Core approach: one grammar connects everything

One shared grammar defines what to generate, what to check, what to repair, and what to measure.

Grammar template

{prefix} exactly {number} {object_1_plural} {relation_text} a {object_2} {scene_context}.

Prompt instance

A realistic image of exactly 4 dice inside a cup on a shelf full of books and decorations.

Atomic constraints

- object identity: dice exist?
- object identity: cup exists?
- cardinality: count = 4?
- spatial relation: dice inside cup?

Example slot values: number = 4, obj1 = dice, relation = inside, obj2 = cup, context = shelf with books/decorations

Labels (pass / fail / ambiguous)

dice identity: pass
cup identity: pass
count: fail
spatial relation: pass

Repair (Original prompt + targeted instruction)

Make sure there are exactly 4 dice; do not generate more or fewer, and keep them separated and countable.

Metrics

constraint pass/fail/ambiguous

image pass = all constraints pass

repair success = non-pass → pass

regression = pass → non-pass

Constraint design: labelable and repairable requirements

A realistic image of exactly 4 dice inside a glossy white cup on a shelf full of books and decorations.

Four representative constraint types used in this project

- Object identity
  - clearly identifiable dice?
  - clearly identifiable cup?
- Cardinality
  - exactly 4 dice?
- Attribute
  - cup glossy white?
- Spatial relation
  - dice inside the cup?

Labeling rule

- pass = good enough for the prompt
- fail = clearly wrong / useful repair signal
- ambiguous = uncertain or not confidently judgeable

Ambiguous includes generated object-like shapes whose identity is uncertain, not only blur or occlusion.

Experimental workflow: from prompt to constraint-level repair

Original prompt

A realistic image of exactly 4 dice inside a cup on a shelf full of books and decorations.

Initial image

Constraint labels

dice identity: pass
cup identity: pass
count: fail
spatial relation: pass

Repair prompt

Original prompt + "Important generation instructions:

- Make sure that there are exactly 4 dice; do not generate more or fewer, and do not add extra target-like objects."

Repaired image

Final experiment setting

2 T2I models:
OpenAI gpt-image-1, Gemini 2.5 Flash Image

180 prompts / 720 constraints

Human labels = ground truth
GPT-4.1 VLM checker = automatic repair trigger
Repair settings: human-triggered / VLM-triggered

Initial checking results: partial failures + VLM agreement

HUMAN LABELS SHOW PARTIAL FAILURES

55.0%
image-level human pass

85.8%
constraint-level human pass

Many images fail as a whole, while most individual constraints are satisfied.

VLM CHECKER VS HUMAN LABELS

Level	Agreement	Non-pass recall	Non-pass precision
Image	**69.4%**	**82.7%**	**72.0%**
Constraint	**80.4%**	**71.6%**	**42.9%**

The VLM is not ground truth, but it catches many human non-pass cases and can trigger repair automatically.

Takeaway: constraint-level labels make partial failures visible; VLM checking is usable as a practical repair trigger, but human labels remain the evaluation ground truth.

Repair results: constraint-aware prompts improve correctness

BEFORE/AFTER REPAIR UNDER HUMAN EVALUATION

Trigger	Attempts	Image pass Δ	Constraint pass Δ	Image fixed	Regression
Human	81	**+51.9 pts**	**+16.7 pts**	**51.9%**	**11.1%**
VLM	93	**+28.0 pts**	**+9.7 pts**	**34.4%**	**15.1%**
Combined	174	**+39.1 pts**	**+12.9 pts**	**42.5%**	**13.2%**

Human-triggered
Oracle setting: true failed constraints are known.

VLM-triggered
Practical setting: automatic checker chooses repair targets.

Combined
Overall behavior across all repair attempts.

Both oracle and practical settings improve human-evaluated correctness; VLM-triggered repair is noisier but still useful.

What did the constraint-level analysis reveal?

Combined prompts are harder	Single prompt families ≈ 67–71% image pass; combined families ≈ 36–53%
Cardinality is the hardest initial constraint	Initial pass: cardinality 58.3%, attribute 94.3%, spatial 89.6%, object identity 87.8%
Repair helps unevenly	Combined repair non-pass→pass: attribute 95.0%, spatial 78.9%, object identity 71.2%, cardinality 44.7%

Constraint-level structure is useful because it shows not only whether repair works, but where it works and where it still struggles.